

0.84. Des vies et une fin de partie

Le poursuivant du 0.79, et 3 vies. Attrapé : une vie de moins, et tout le monde repart. À 0 : PERDU, et START pour recommencer.

1. Ce qu'il faut comprendre

C'est le poursuivant du 0.79, avec des vies. On en a 3. Chaque fois que le B nous attrape, on en perd une, et le A comme le B **repartent de leur place de départ**.

Ce qui est nouveau ici : l'ÉTAT du jeu. La variable `etat` dit **où on en est** : 0, on joue ; 1, on a perdu. Tout le programme est coupé en deux par `if (etat == 0) { ... } else { ... }` : en jeu, on bouge ; perdu, on attend.

Perdu : quand `vies` arrive à 0, `etat = 1;` et le mot PERDU s'écrit au milieu. Dans l'état 1, plus rien ne bouge.

Recommencer : dans l'état 1, le bouton **START** remet 3 vies et `etat = 0;` : on rejoue.

Repartir du départ après une prise : on efface la case (le A et le B y sont tous les deux), puis on remet `x`, `y`, `bx`, `by` à leurs valeurs de départ.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2600 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES ----
```

2. UNE VARIABLE — X

```
uint8_t x = 10;      // le A : il part du milieu, (10, 8)
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — BX

```
uint8_t bx = 0;      // le B, le poursuivant : il part du coin, (0, 0)
```

5. UNE VARIABLE — BY

```
uint8_t by = 0;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;      // la direction du prochain pas du B (le 0.79)
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

8. UNE VARIABLE — VIES

```
uint8_t vies = 3;    // les vies qui restent : 3 au départ
```

9. UNE VARIABLE — ETAT

```
uint8_t etat = 0;    // L'ÉTAT DU JEU : 0 = on joue, 1 = perdu
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(0, 17, "VIES");                // le mot VIES, en bas à gauche

    while (true) {
        image();

        // ---- LE PROGRAMME EST COUPÉ EN DEUX PAR L'ÉTAT ----
        //
        //  if (etat == 0) { ... on joue ... } else { ... perdu : on attend START ... }
        //
        // Un seul des deux blocs se fait à chaque image, jamais les deux.
        if (etat == 0) {                  // ===== ON JOUE

            // Le B avance vers le A, un pas tous les 600 ms (le 0.79).
            if (chaque(600)) {
                sx = 0;
                if (x > bx) { sx = 1; }    // le A est à droite : un pas à droite
                if (x < bx) { sx = -1; }   // à gauche : un pas à gauche
                sy = 0;
                if (y > by) { sy = 1; }    // plus bas : un pas en bas
                if (y < by) { sy = -1; }   // plus haut : un pas en haut
                if (sx != 0 || sy != 0) {  // pas déjà sur lui
                    un_pas(bx, by, ALPHABET[1], sx, sy);
                }
            }

            deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix

            // ATTRAPÉ : le B est sur la case du A.
            if (bx == x && by == y) {
                vies = vies - 1;           // 1. une vie de moins : 3 → 2 → 1 → 0
                effacer(x, y, 1);          // 2. efface la case (le A et le B y sont)
                x = 10;                    // 3. tout le monde repart du départ :
                y = 8;                      // le A au milieu...
                bx = 0;                     // ...le B dans le coin
                by = 0;

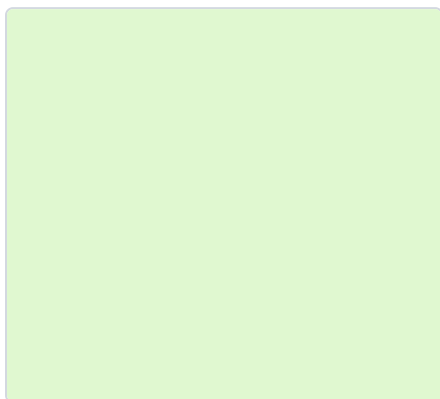
                if (vies == 0) {            // 4. plus aucune vie ?
                    etat = 1;               // on passe dans l'état « perdu »...
                    texte(7, 8, "PERDU");  // ...et on l'écrit au milieu
                }
            }

            nombre(5, 17, vies);           // les vies, après le mot VIES

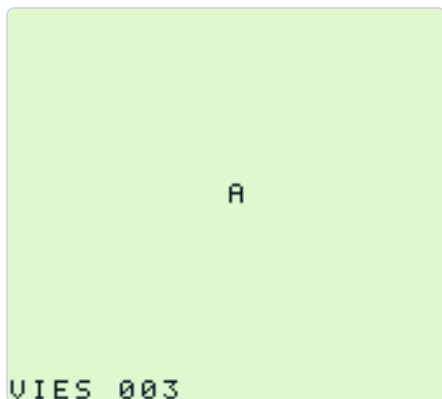
        } else {                          // ===== PERDU
            // Plus rien ne bouge : on attend seulement le bouton START.
            if (bouton(START)) {
                effacer(7, 8, 5);          // efface les 5 lettres de PERDU
                vies = 3;                   // de nouveau 3 vies...
                etat = 0;                   // ...et on rejoue
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

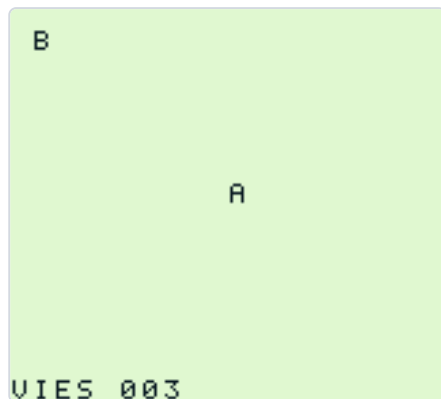
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



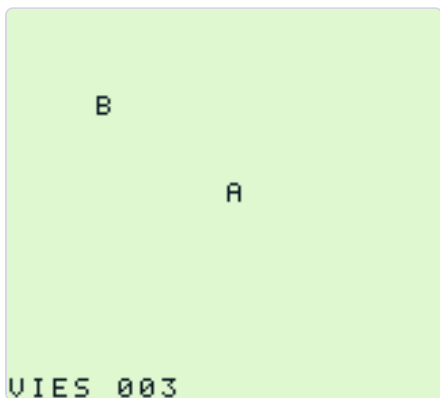
la toute première image — la console vient de démarrer — image 1



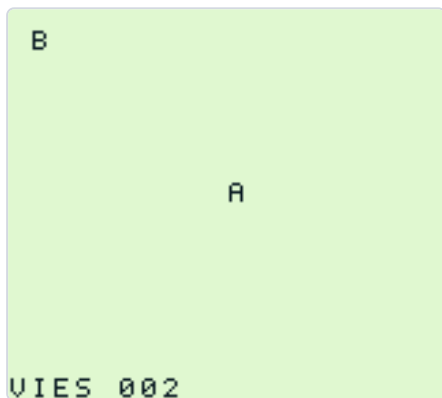
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



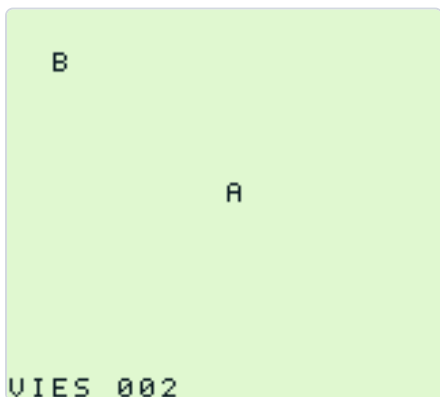
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

Le B poursuit le A ; à chaque prise, une vie de moins et tout repart. Au bout de trois : PERDU. START relance la partie.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ immobile, on perd les 3 vies : PERDU
 - ✓ START fait recommencer : 3 vies, en jeu
 - ✓ PERDU est effacé
-

Ensuite : **0.85. Plusieurs ennemis : un tableau de struct** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++